

Prototipo Zoom

Sistema de Videoconferencia por Sockets

PC4 - SW403U - Programación Orientada a Objetos

Universidad Nacional de Ingeniería

Facultad de Ingeniería Industrial y Sistemas

Integrantes:

Yampufe Pau Héctor Alonso - 20240310I

Romaní Cuyo Gabriel Alberto - 20242045K

Tasayco Portello Luis Antonio - 20240307H

Docente: CISNEROS NAPRAVNIK YAN EDUARDO

24 de junio de 2026

Índice

1. Introducción

El presente documento describe el diseño e implementación de un **prototipo académico de videoconferencia** inspirado en Zoom, desarrollado en Python como parte del curso de Programación Orientada a Objetos (PC4).

El sistema emplea una arquitectura **cliente-servidor** basada en **sockets TCP** con un protocolo de mensajes en formato **JSON**. Cuenta con autenticación de usuarios, creación y gestión de salas virtuales, chat en tiempo real, transferencia de archivos y transmisión de video desde la cámara web.

2. Objetivos

- Aplicar los principios de la POO: herencia, encapsulamiento, polimorfismo.
- Implementar una arquitectura cliente-servidor mediante sockets TCP.
- Desarrollar un protocolo de comunicación propio basado en JSON.
- Integrar múltiples funcionalidades: login, salas, chat, archivos y video.
- Utilizar SQLite para la persistencia de información.
- Emplear concurrencia con **threading** para atender múltiples clientes.

3. Tecnologías Utilizadas

- **Lenguaje:** Python 3.10+
- **Interfaz gráfica:** Tkinter
- **Comunicación:** Sockets TCP
- **Protocolo:** JSON sobre TCP
- **Base de datos:** SQLite
- **Concurrencia:** threading
- **Video:** OpenCV o ffmpeg
- **Imágenes:** Pillow

4. Arquitectura del Sistema

El sistema sigue un modelo cliente-servidor. El servidor centraliza las conexiones y actúa como intermediario. Cada cliente se conecta mediante TCP y el servidor asigna un hilo independiente por conexión.

5. Base de Datos

La base de datos SQLite contiene las siguientes tablas:

Tabla	Columnas
Usuarios	IdUsuario, Nombres, Correo, PasswordHash, Rol, Activo
Salas	IdSala,CodigoSala, Nombre, IdHost, Estado, FechaCreacion
ParticipantesSala	IdParticipante, IdSala, IdUsuario, Estado, FechaIngreso
SolicitudesSala	IdSolicitud, IdSala, IdUsuario, Estado, FechaSolicitud
Mensajes	IdMensaje, IdSala, IdUsuario, Contenido, FechaEnvio
ArchivosCompartidos	IdArchivo, IdSala, IdUsuario, NombreArchivo, RutaArchivo, Tamano, FechaSubida

Cuadro 1: Tablas de la base de datos

6. Diagrama de Casos de Uso

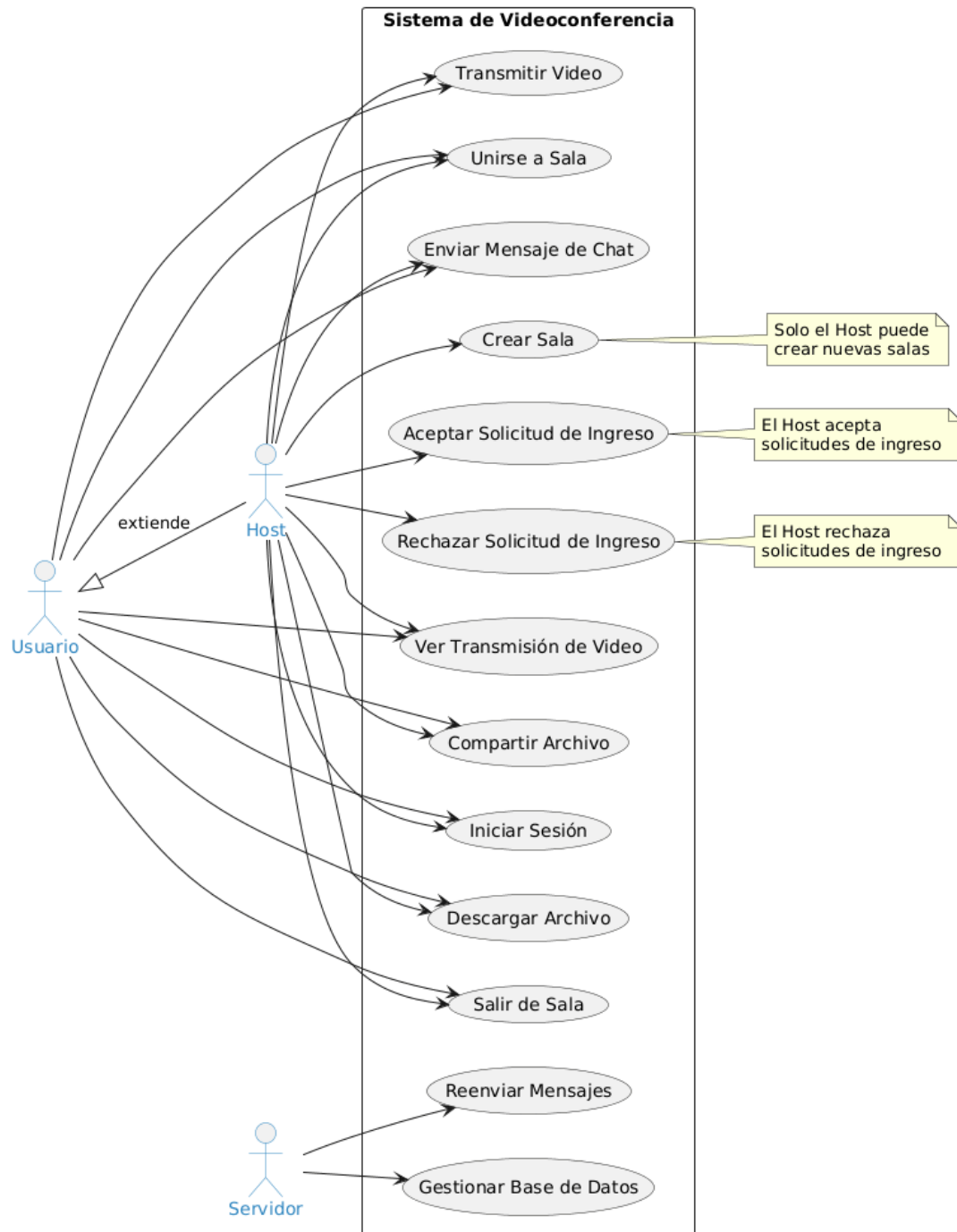


Figura 1: Diagrama de casos de uso del sistema

7. Diagrama de Secuencia

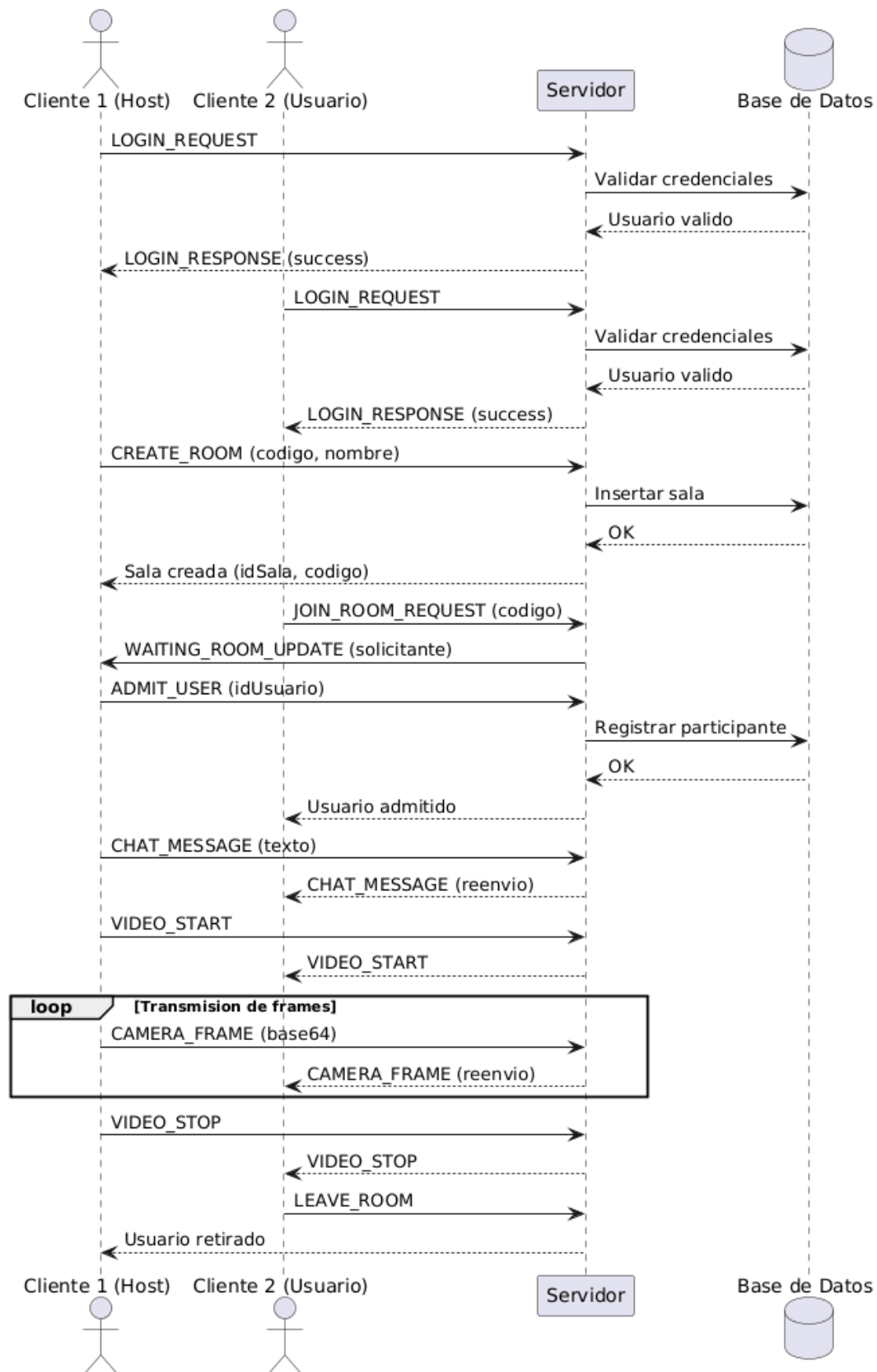
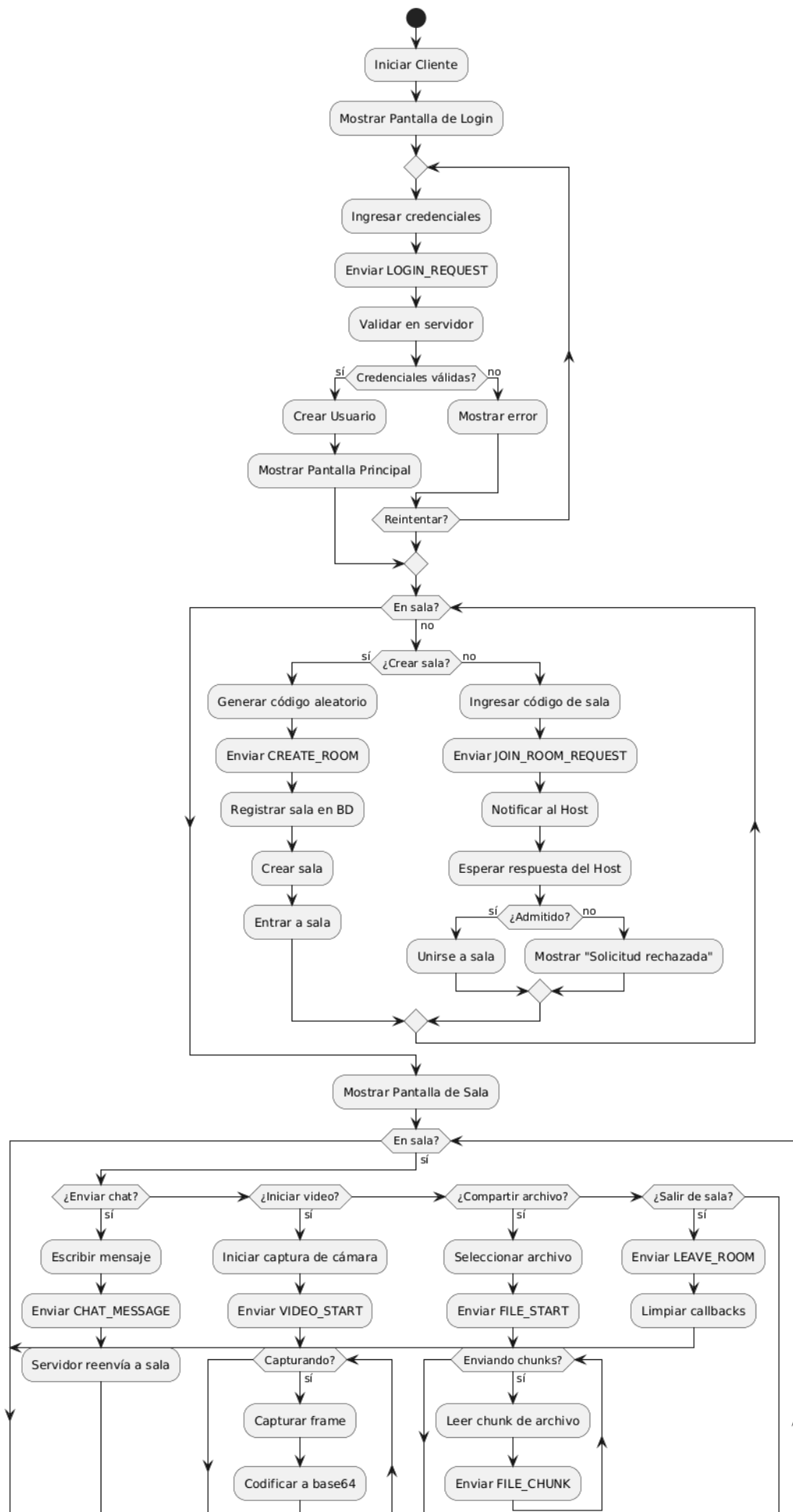


Figura 2: Diagrama de secuencia del flujo de la aplicación

8. Diagrama de Actividades



9. Diagrama de Clases

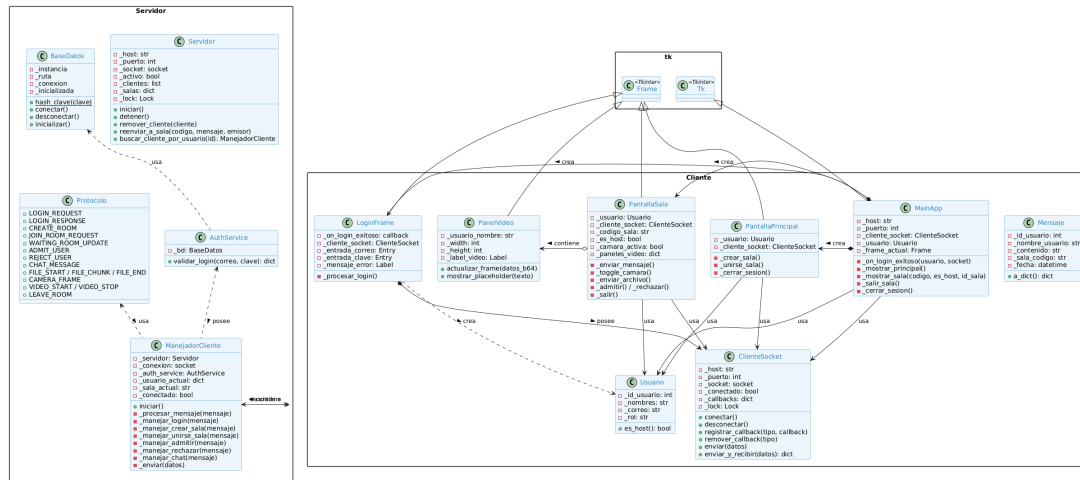


Figura 4: Diagrama de clases del sistema

10. Descripción de Clases

10.1. Lado del Servidor

10.1.1. Protocolo

Servidor/protocolo.py

Clase de constantes que define todos los tipos de mensaje del protocolo:

```

1 class Protocolo:
2     LOGIN_REQUEST = "LOGIN_REQUEST"
3     LOGIN_RESPONSE = "LOGIN_RESPONSE"
4     CREATE_ROOM = "CREATE_ROOM"
5     JOIN_ROOM_REQUEST = "JOIN_ROOM_REQUEST"
6     CHAT_MESSAGE = "CHAT_MESSAGE"
7     CAMERA_FRAME = "CAMERA_FRAME"
8     FILE_START = "FILE_START"
9     FILE_CHUNK = "FILE_CHUNK"
10    FILE_END = "FILE_END"
11    LEAVE_ROOM = "LEAVE_ROOM"

```

10.1.2. BaseDatos

Servidor/base_datos.py

Singleton que gestiona la conexión a SQLite. Inicializa el esquema y los datos de prueba.

```
1 class BaseDatos:
2     _instancia = None
3
4     def __new__(cls, *args, **kwargs):
5         if cls._instancia is None:
6             cls._instancia = super().__new__(cls)
7         return cls._instancia
8
9     def inicializar(self):
10        with open("crear_tablas.sql") as f:
11            self.conectar().cursor().executescript(f.read())
```

10.1.3. AuthService

Servidor/auth_service.py

Valida credenciales consultando la BD con hash SHA-256:

```
1 class AuthService:
2     def validar_login(self, correo, clave):
3         password_hash = BaseDatos.hash_clave(clave)
4         cursor.execute(
5             "SELECT IdUsuario, Nombres, Rol FROM Usuarios"
6             "WHERE Correo=? AND PasswordHash=? AND Activo=?1",
7             (correo, password_hash)
8         )
9         resultado = cursor.fetchone()
10        if resultado:
11            return {"status": "success", "idUsuario": ...}
12        return {"status": "error", "message": "Credenciales incorrectas"
13            }
```

10.1.4. ManejadorCliente

Servidor/manejador_cliente.py

Maneja la comunicación con un cliente en un hilo separado. El bucle principal lee mensajes JSON delimitados por \n y los despacha según su tipo:

```
1 class ManejadorCliente:
2     def iniciar(self):
3         buffer = ""
4         while self._conectado:
5             datos = self._conexion.recv(4096).decode("utf-8")
6             buffer += datos
7             while "\n" in buffer:
8                 linea, buffer = buffer.split("\n", 1)
9                 mensaje = json.loads(linea)
10                self._procesar_mensaje(mensaje)
11
12    def _procesar_mensaje(self, mensaje):
13        tipo = mensaje.get("type")
14        if tipo == Protocolo.LOGIN_REQUEST:
15            self._manejar_login(mensaje)
16        elif tipo == Protocolo.CREATE_ROOM:
17            self._manejar_crear_sala(mensaje)
18        elif tipo == Protocolo.CHAT_MESSAGE:
19            self._servidor.reenviar_a_sala(
20                mensaje.get("roomCode"), mensaje, self)
```

10.1.5. Servidor

Servidor/servidor.py

Servidor TCP principal. Acepta conexiones y crea un hilo por cada cliente:

```
1 class Servidor:
2     def iniciar(self):
3         BaseDatos().inicializar()
4         self._socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5         self._socket.bind((self._host, self._puerto))
6         self._socket.listen()
7         while self._activo:
8             conexion, direccion = self._socket.accept()
9             manejador = ManejadorCliente(self, conexion, direccion)
10            hilo = threading.Thread(target=manejador.iniciar, daemon=
11                True)
12            hilo.start()
```

10.2. Lado del Cliente

10.2.1. MainApp

Cliente/main.py

Ventana principal Tkinter. Gestiona la navegación entre pantallas:

```
1 class MainApp(tk.Tk):
2     def _on_login_exitoso(self, usuario, cliente_socket):
3         self._usuario = usuario
4         self._cliente_socket = cliente_socket
5         self._mostrar_principal()
6
7     def _mostrar_sala(self, codigo_sala, es_host, id_sala):
8         self.geometry("900x600")
9         self._frame_actual = PantallaSala(
10             self, self._usuario, self._cliente_socket,
11             codigo_sala, es_host, id_sala, self._salir_sala
12         )
13         self._frame_actual.pack(fill=tk.BOTH, expand=True)
```

10.2.2. ClienteSocket

Cliente/cliente_socket.py

Abstracción del socket TCP con callbacks para despachar mensajes entrantes:

```
1 class ClienteSocket:
2     def _escuchar(self):
3         buffer = ""
4         while self._conectado:
5             datos = self._socket.recv(4096).decode("utf-8")
6             buffer += datos
7             while "\n" in buffer:
8                 linea, buffer = buffer.split("\n", 1)
9                 mensaje = json.loads(linea)
10                 tipo = mensaje.get("type")
11                 cb = self._callbacks.get(tipo)
12                 if cb:
13                     cb(mensaje)
```

```
15     def enviar(self, datos):
16         self._socket.send(
17             (json.dumps(datos) + "\n").encode("utf-8"))
```

10.2.3. Usuario

Cliente/modelos/usuario.py

Modelo del usuario autenticado:

```
1 class Usuario:
2     def __init__(self, id_usuario, nombres, correo, rol):
3         self._id_usuario = id_usuario
4         self._nombres = nombres
5         self._correo = correo
6         self._rol = rol
7
8     def es_host(self):
9         return self._rol.lower() == "host"
```

10.2.4. Mensaje

Cliente/modelos/mensaje.py

Modelo de mensaje de chat con serialización a diccionario:

```
1 class Mensaje:
2     def a_dict(self):
3         return {
4             "type": "CHAT_MESSAGE",
5             "roomCode": self._sala_codigo,
6             "userId": self._id_usuario,
7             "userName": self._nombre_usuario,
8             "message": self._contenido
9         }
```

10.2.5. LoginFrame

Cliente/pantallas/login.py

Pantalla de inicio de sesión. Envía LOGIN_REQUEST al servidor:

```
1 class LoginFrame(tk.Frame):
2     def _procesar_login(self):
3         respuesta = self._cliente_socket.enviar_y_recibir({
4             "type": "LOGIN_REQUEST",
5             "correo": self._entrada_correo.get(),
6             "clave": self._entrada_clave.get()
7         })
8         if respuesta.get("status") == "success":
9             usuario = Usuario(respuesta["idUsuario"],
10                                respuesta["nombres"], correo,
11                                respuesta["rol"])
12             self._on_login_exitoso(usuario, self._cliente_socket)
```

10.2.6. PantallaPrincipal

Cliente/pantallas/pantalla_principal.py

Menú principal con opciones de crear o unirse a una sala:

```
1 class PantallaPrincipal(tk.Frame):
2     def _crear_sala(self):
3         codigo = ''.join(random.choices(
4             string.ascii_uppercase + string.digits, k=6))
5         respuesta = self._cliente_socket.enviar_y_recibir({
6             "type": "CREATE_ROOM",
7             "nombre": f"Sala_{self._usuario.nombres}",
8             "codigo": codigo,
9             "idHost": self._usuario.id_usuario
10        })
11        if respuesta.get("status") == "success":
12            self._on_entrar_sala(codigo, True, respuesta["idSala"])
```

10.2.7. PantallaSala

Cliente/pantallas/pantalla_sala.py

Pantalla principal de la sala. Integra chat, video y transferencia de archivos:

```
1 class PantallaSala(tk.Frame):
2     def _enviar_mensaje(self):
3         texto = self._entrada_msg.get().strip()
```

```
4         self._cliente_socket.enviar({
5             "type": "CHAT_MESSAGE",
6             "roomCode": self._codigo_sala,
7             "userName": self._usuario.nombres,
8             "message": texto
9         })
10
11     def _enviar_archivo_thread(self, ruta, nombre, tamano):
12         with open(ruta, "rb") as f:
13             datos = base64.b64encode(f.read()).decode()
14             self._cliente_socket.enviar({
15                 "type": "FILE_START", "roomCode": self._codigo_sala,
16                 "fileName": nombre, "fileSize": tamano
17             })
18             for i in range(0, len(datos), 1500):
19                 self._cliente_socket.enviar({
20                     "type": "FILE_CHUNK", "roomCode": self._codigo_sala,
21                     "fileName": nombre, "data": datos[i:i+1500]
22                 })
```

10.2.8. PanelVideo

Cliente/pantallas/pantalla_video.py

Panel que muestra frames de video desde base64:

```
1 class PanelVideo(tk.Frame):
2     def actualizar_frame(self, datos_b64):
3         img_data = base64.b64decode(datos_b64)
4         img = Image.open(io.BytesIO(img_data))
5         img = img.resize((self._width, self._height), Image.LANCZOS)
6         self._imagen_tk = ImageTk.PhotoImage(img)
7         self._label_video.config(image=self._imagen_tk, text="")
```

11. Pilares de la Programación Orientada a Objetos

El sistema aplica los cuatro pilares fundamentales de la POO, detallados a continuación con fragmentos de código extraídos del proyecto.

11.1. Encapsulamiento

Todas las clases emplean atributos privados (prefijo `_`) y exponen únicamente lo necesario mediante propiedades de solo lectura o métodos públicos.

Ejemplo: Cliente/modelos/usuario.py

```
1 class Usuario:
2     def __init__(self, id_usuario, nombres, correo, rol):
3         self._id_usuario = id_usuario
4         self._nombres = nombres
5         self._correo = correo
6         self._rol = rol
7
8     @property
9     def id_usuario(self):
10         return self._id_usuario
11
12     @property
13     def nombres(self):
14         return self._nombres
15
16     def es_host(self):
17         return self._rol.lower() == "host"
```

Los atributos `_id_usuario`, `_nombres`, `_correo` y `_rol` solo son accesibles mediante `@property`. La lógica de verificación de rol queda encapsulada en `es_host()`.

Ejemplo: Servidor/servidor.py

```
1 class Servidor:
2     def __init__(self, host="0.0.0.0", puerto=5000):
3         self._host = host
4         self._puerto = puerto
5         self._socket = None
6         self._activo = False
7         self._clientes = []
8         self._salas = {}
9         self._lock = threading.Lock()
10
11     def iniciar(self):
12         ...
```



```
13     def detener(self):
14         ...
15     def reenviar_a_sala(self, codigo_sala, mensaje, emisor=None):
16         ...
```

El estado interno del servidor es privado; la interacción externa ocurre solo a través de los métodos públicos `iniciar()`, `detener()`, `remover_cliente()` y `reenviar_a_sala()`.

11.2. Herencia

Las clases de la interfaz gráfica heredan de componentes Tkinter para reutilizar su funcionalidad y extenderla con lógica de negocio.

Ejemplo: Cliente/main.py

```
1 class MainApp(tk.Tk):                                # Herencia de ventana principal
2     def __init__(self, host, puerto):
3         super().__init__()
4         ...
5         self._frame_actual = None
6         self._mostrar_login()
```

Ejemplo: Cliente/pantallas/pantalla_video.py

```
1 class PanelVideo(tk.Frame):                            # Herencia de Frame
2     def __init__(self, master, usuario_nombre="", width=240, height=180,
3         bg="#222"):
4         super().__init__(master, bg=bg, width=width, height=height)
5         self.pack_propagate(False)
6         ...
```

Otras clases que siguen el mismo patrón: `LoginFrame(tk.Frame)`, `PantallaPrincipal(tk.Frame)` y `PantallaSala(tk.Frame)`. Todas heredan los métodos de gestión de geometría y el bucle de eventos de Tkinter.

11.3. Polimorfismo

El polimorfismo se manifiesta en tres áreas clave del sistema.

Frames intercambiables: En `MainApp`, la variable `self._frame_actual` puede contener cualquier subclase de `tk.Frame` (`LoginFrame`, `PantallaPrincipal`, `PantallaSala`) y se invocan métodos comunes como `pack()` y `destroy()` sin conocer el tipo concreto.

```
1 class MainApp(tk.Tk):
2     def _mostrar_login(self):
3         if self._frame_actual:
4             self._frame_actual.destroy()
5             self._frame_actual = LoginFrame(self, ...)
6             self._frame_actual.pack(fill=tk.BOTH, expand=True)
7
8     def _mostrar_principal(self):
9         if self._frame_actual:
10             self._frame_actual.destroy()
11             self._frame_actual = PantallaPrincipal(self, ...)
12             self._frame_actual.pack(fill=tk.BOTH, expand=True)
```

Backends de cámara: Los métodos `_capturar_con_opencv()` y `_capturar_con_ffmpeg()` implementan la misma interfaz conceptual y se seleccionan en tiempo de ejecución.

```
1 def _capturar_y_enviar(self):
2     if self._backend_video == "opencv":
3         self._capturar_con_opencv()
4     else:
5         self._capturar_con_ffmpeg()
```

Despacho de mensajes: El servidor procesa todos los mensajes entrantes con un único método que deriva cada tipo a su manejador específico.

```
1 def _procesar_mensaje(self, mensaje):
2     tipo = mensaje.get("type")
3     if tipo == Protocolo.LOGIN_REQUEST:
4         self._manejar_login(mensaje)
5     elif tipo == Protocolo.CREATE_ROOM:
6         self._manejar_crear_sala(mensaje)
7     elif tipo == Protocolo.CHAT_MESSAGE:
8         self._manejar_chat(mensaje)
9     elif tipo == Protocolo.LEAVE_ROOM:
10         self._sala_actual = None
```

11.4. Abstracción

Cada clase compleja expone una interfaz reducida que oculta los detalles de implementación subyacentes.

Ejemplo: Cliente/cliente_socket.py

```
1 class ClienteSocket:
2     def conectar(self): ...
3     def desconectar(self): ...
4     def enviar(self, datos): ...
5     def enviar_y_recibir(self, datos): ...
6     def registrar_callback(self, tipo, callback): ...
7     def remover_callback(self, tipo): ...
```

Esta clase abstrae toda la complejidad de la comunicación por sockets TCP: creación del socket, serialización JSON, threading para escucha continua, buffer de mensajes y dispatch mediante callbacks.

Ejemplo: Servidor/base_datos.py

```
1 class BaseDatos:
2     def conectar(self): ...
3     def desconectar(self): ...
4     def inicializar(self): ...
5     @staticmethod
6     def hash_clave(clave): ...
```

Abstrae SQLite (creación de tablas, conexión, hashing de contraseñas) en solo cuatro operaciones.

Ejemplo: Servidor/auth_service.py

```
1 class AuthService:
2     def validar_login(self, correo, clave):
3         password_hash = BaseDatos.hash_clave(clave)
4         cursor.execute(
5             "SELECT IdUsuario, Nombres, Rol FROM Usuarios"
6             "WHERE Correo=? AND PasswordHash=? AND Activo=1",
7             (correo, password_hash)
8         )
9         resultado = cursor.fetchone()
10        if resultado:
11            return {"status": "success", ...}
```

```
12     return {"status": "error", "message": "Credenciales incorrectas"}
    }
```

Ocultas las consultas SQL y el hashado detrás de un único método `validar_login()`. Quien lo invoca no necesita conocer la estructura de la base de datos.

Ejemplo: Cliente/pantallas/pantalla_video.py

```
1 class PanelVideo(tk.Frame):
2     def actualizar_frame(self, datos_b64): ...
3     def mostrar_placeholder(self, texto="Sin video"): ...
```

Ocultas los detalles de PIL/Tkinter (decodificación base64, redimensión, conversión a `PhotoImage`) detrás de una interfaz de dos métodos.

12. Patrones de Diseño Implementados

A continuación se describen los patrones de diseño identificados en la implementación del sistema.

12.1. Singleton

Propósito: Garantizar que una clase tenga una única instancia y proporcionar un punto de acceso global a ella.

Implementación: Servidor/base_datos.py

La clase `BaseDatos` sobrescribe `__new__` para retornar siempre la misma instancia. La inicialización del esquema se ejecuta una sola vez mediante una bandera `_inicializada`.

```
1 class BaseDatos:
2     _instancia = None
3
4     def __new__(cls, *args, **kwargs):
5         if cls._instancia is None:
6             cls._instancia = super().__new__(cls)
7         return cls._instancia
```

Todas las llamadas a `BaseDatos()` desde `manejador_cliente.py` y `auth_service.py` obtienen la misma instancia única.

12.2. Observer

Propósito: Definir una dependencia de uno a muchos entre objetos, de modo que cuando un objeto cambie su estado, todos sus dependientes sean notificados automáticamente.

Implementación: Cliente/cliente_socket.py

La clase `ClienteSocket` mantiene un diccionario `_callbacks` que asocia tipos de mensaje con funciones manejadoras. Cuando llega un mensaje desde el servidor, se invoca la función registrada para ese tipo:

```
1 class ClienteSocket:
2     def registrar_callback(self, tipo, callback):
3         self._callbacks[tipo] = callback
4
5     def _escuchar(self):
6         while self._conectado:
7             mensaje = json.loads(linea)
8             tipo = mensaje.get("type")
9             cb = self._callbacks.get(tipo)
10            if cb:
11                cb(mensaje)
```

Las pantallas (`PantallaPrincipal`, `PantallaSala`) se suscriben como observadores mediante `registrar_callback()` y se desuscriben al salir con `remove_callback()`.

12.3. Strategy

Propósito: Definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables.

Implementación: Cliente/pantallas/pantalla_sala.py

El sistema selecciona en tiempo de ejecución entre dos estrategias de captura de video: `_capturar_con_opencv()` y `_capturar_con_ffmpeg()`. El método `_detectar_backend()` elige la estrategia disponible y `_capturar_y_enviar()` delega en ella:

```
1 def _detectar_backend(self):
2     try:
3         import cv2
4         self._backend_video = "opencv"
5     except ImportError:
```

```
6         self._backend_video = "ffmpeg"
7
8     def _capturar_y_enviar(self):
9         if self._backend_video == "opencv":
10             self._capturar_con_opencv()
11         else:
12             self._capturar_con_ffmpeg()
```

12.4. Facade

Propósito: Proporcionar una interfaz unificada para un conjunto de interfaces de un subsistema.

Implementación: Servidor/manejador_cliente.py

ManejadorCliente actúa como una fachada que orquesta tres subsistemas: AuthService (autenticación), BaseDatos (persistencia) y Servidor (difusión en salas). El exterior (servidor.py) solo invoca manejador.iniciar(), y la fachada se encarga de despachar cada mensaje al subsistema correspondiente.

12.5. Command

Propósito: Encapsular una solicitud como un objeto, permitiendo parametrizar clientes con distintas solicitudes.

Implementación: Servidor/protocolo.py y Servidor/manejador_cliente.py

El protocolo define constantes (LOGIN_REQUEST, CREATE_ROOM, JOIN_ROOM_REQUEST, etc.) que actúan como comandos. ManejadorCliente._procesar_mensaje() funciona como despachador: lee el campo type del mensaje JSON y lo enruta al manejador correspondiente.

12.6. Template Method

Propósito: Definir el esqueleto de un algoritmo en una operación, delegando algunos pasos a las subclases.

Implementación: Cliente/pantallas/pantalla_sala.py

El ciclo de vida de la cámara sigue un algoritmo fijo: iniciar captura, capturar y enviar fotogramas, detener captura. Los pasos concretos (`_capturar_con_opencv()` o `_capturar_con_ffmpeg()`) se inyectan en ese esqueleto.

12.7. Factory Method

Propósito: Definir una interfaz para crear un objeto, pero dejar que las subclases decidan qué clase instanciar.

Implementación: Cliente/main.py

MainApp implementa métodos factoría (`_mostrar_login()`, `_mostrar_principal()`, `_mostrar_sala()`) que destruyen el frame actual e instancian la subclase de `tk.Frame` correspondiente:

```
1 def _mostrar_login(self):
2     if self._frame_actual:
3         self._frame_actual.destroy()
4     self._frame_actual = LoginFrame(self, self._conectar)
5     self._frame_actual.pack(fill=tk.BOTH, expand=True)
```

13. Protocolo de Comunicación

Los mensajes se envían en JSON sobre TCP, delimitados por `\n`.

14. Manual de Instalación y Ejecución

14.1. Requisitos

- Python 3.10 o superior
- Pillow (instalar con `pip install -r requirements.txt`)
- Opcional: OpenCV (`pip install opencv-python`) o ffmpeg

14.2. Ejecución

1. Iniciar el servidor:

Tipo	Dirección	Campos
LOGIN_REQUEST	Cliente → Servidor	correo, clave
LOGIN_RESPONSE	Servidor → Cliente	status, idUsuario, nombres, rol
CREATE_ROOM	Cliente → Servidor	codigo, nombre, idHost
JOIN_ROOM_REQUEST	Cliente → Servidor	codigo, idUsuario
WAITING_ROOM_UPDATE	Servidor → Host	idSala, solicitanteId, solicitanteNombre
ADMIT_USER	Host → Servidor	idSala, idUsuario
REJECT_USER	Host → Servidor	idSala, idUsuario
CHAT_MESSAGE	Cliente → Sala	roomCode, userName, message
FILE_START	Cliente → Sala	roomCode, fileName, fileSize
FILE_CHUNK	Cliente → Sala	roomCode, fileName, data
FILE_END	Cliente → Sala	roomCode, fileName
CAMERA_FRAME	Cliente → Sala	roomCode, userName, data (base64)
VIDEO_START	Cliente → Sala	roomCode, userName
VIDEO_STOP	Cliente → Sala	roomCode, userName
LEAVE_ROOM	Cliente → Servidor	roomCode

Cuadro 2: Tipos de mensaje del protocolo


```
1 python -m Servidor.servidor
```

2. Iniciar uno o más clientes:

```
1 python -m Cliente.main
```

14.3. Datos de Prueba

Nombre	Correo	Clave	Rol
Luis Tasayco	luis@email.com	123456	Host
Ana Torres	ana@email.com	123456	Usuario
Carlos Ruiz	carlos@email.com	123456	Usuario

Cuadro 3: Usuarios de prueba

15. Conclusiones

- Se implementó un prototipo funcional de videoconferencia con arquitectura cliente-servidor.
- Los sockets TCP con protocolo JSON proporcionan una base sólida para comunicación en tiempo real.
- Los cuatro pilares de la POO – encapsulamiento, herencia, polimorfismo y abstracción – fueron aplicados consistentemente en todo el código.
- Se identificaron y aplicaron siete patrones de diseño: Singleton, Observer, Strategy, Facade, Command, Template Method y Factory Method.
- La POO permitió organizar el código en clases especializadas con responsabilidades claras.
- La concurrencia con hilos permite atender múltiples clientes simultáneamente.
- La transmisión de video se logró mediante captura, compresión JPEG y codificación base64.